Coding Challenge Submission Report

Name of student: Mrunali Tarkesh Jibhakate

Roll No / ID: 107 / 23071350

Class / Semester: 3 yr / 6th sem

Date of Submission: 13/03/2026

Problem Details

Problem Title:

1. Longest Common Prefix
2. Minimum Path Sum
3. Add Binary
4. Text Justification

Problem Statement (as given):

1. Write a function to find the longest common prefix string amongst an array of strings
2. Given a m x n grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.
3. Given two binary strings a and b, return their sum as a binary string.
4. Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified.You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters.Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

Difficulty Level:

1. Easy
2. Medium
3. Easy
4. Hard

LeetCode Link:

1. https://leetcode.com/problems/longest-common-prefix/description/

2. https://leetcode.com/problems/minimum-path-sum/description/
3. https://leetcode.com/problems/add-binary/description/
4. https://leetcode.com/problems/text-justification/description/

Student's Solution

Approach / Logic Used:

1.

```
char* longestCommonPrefix(char** strs, int strsSize)
{
    if (strsSize == 0) return "";
    int minLen = strlen(strs[0]);

    for (int i = 1; i < strsSize; i++)
    {
        int len = strlen(strs[i]);
        if (len < minLen) minLen = len;
    }
    char* prefix = (char*)malloc(sizeof(char) * (minLen + 1));
    int index = 0;

    for (int i = 0; i < minLen; i++)
    {
        char c = strs[0][i];
        for (int j = 1; j < strsSize; j++)
        {
            if (strs[j][i] != c)
            {
                prefix[index] = '\0';
                return prefix;
            }
        }
        prefix[index++] = c;
    }
    prefix[index] = '\0';
    return prefix;
}
```

2.

```
int minPathSum(int ** grid, int gridSize, int * gridColSize)
{
    for (int i = 1; i < *gridColSize; i++)
        grid[0][i] += grid[0][i-1];

    for (int i = 1; i < gridSize; i++)
        grid[i][0] += grid[i-1][0];

    for (int i = 1; i < gridSize; i++)
        for (int j = 1; j < *gridColSize; j++)
            grid[i][j] += (grid[i-1][j] < grid[i][j-1] ? grid[i-1][j] : grid[i][j-1]);

    return grid[gridSize-1][*gridColSize-1];
}
```

3.

```
char* addBinary(char* a, char* b)
{
```

```
    int i = strlen(a) - 1;
    int j = strlen(b) - 1;
    int carry = 0;
    char* res = (char*)malloc(10005);
    int k = 0;
    while (i >= 0 || j >= 0 || carry)
    {
        int total = carry;

        if (i >= 0)
            total += a[i--] - '0';
        if (j >= 0)
            total += b[j--] - '0';
        res[k++] = (total % 2) + '0';
        carry = total / 2;
    }

    res[k] = '\0';
    for (int l = 0, r = k - 1; l < r; l++, r--)
    {
        char tmp = res[l];
        res[l] = res[r];
        res[r] = tmp;
    }

    return res;
}
```

4.

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
char** fullJustify(char** words, int wordsSize, int maxWidth, int* returnSize)
{
    int arr[wordsSize];
    for(int i=0;i<wordsSize;i++)arr[i]=strlen(words[i]);
    int count=0;
    int rows=0;
    for(int i=0;i<wordsSize;i++)
    {

        if(i==0)
        {
            count+=arr[i];
        }
        else if(count+arr[i]+1>maxWidth)
        {
            rows++;
            count=arr[i];
        }
        else
        {
            count=count+arr[i]+1;
        }
    }
    rows++;
    int checker[rows+1];
    checker[0]=-1;
    int ind=1;
    count=0;
    for(int i=0;i<wordsSize;i++)
    {
        if(i==0)
        {
            count+=arr[i];
```

```
    }
    else if(count+arr[i]+1>maxWidth)
    {
        checker[ind++]=i-1;
        count=arr[i];
    }
    else
    {
        count=count+arr[i]+1;
    }
}
checker[ind]=wordsSize-1;
char** res=(char**)calloc(rows,sizeof(char*));
for(int i=0;i<rows;i++)
{
    res[i]=(char*)calloc(maxWidth+1,sizeof(char));
    for(int j=0;j<maxWidth;j++)
    {
        res[i][j]=' ';
    }
}
for(int i=1;i<rows;i++)
{
    int letters=0;
    int spaces=0;
    for(int j=checker[i-1]+1;j<=checker[i];j++)
    {
        letters+=arr[j];
        spaces+=1;
    }
    spaces-=1;
    int itr=maxWidth-1;
    int abc=maxWidth-letters;
    int j;
    for(j=checker[i];j>checker[i-1]+1;j--)
    {
        if(spaces==0)
        {
            int x=0;
            while(x<arr[j])
            {
                res[i-1][x++]=words[j][x++];
            }
        }
        else
        {
            for(int k=arr[j]-1;k>=0;k--)
            {
                res[i-1][itr--]=words[j][k];
            }
            for(int k=0;k<(abc/spaces);k++)
            {
                res[i-1][itr--]=' ';
            }
            abc=abc-(abc/spaces);
            spaces=spaces-1;
        }
    }
    for(int k=0;k<arr[j];k++){
        res[i-1][k]=words[j][k];
    }
}
int itr=0;
for(int j=checker[rows-1]+1;j<=checker[rows];j++)
{
    for(int k=0;k<arr[j];k++)
    {
        res[rows-1][itr++]=words[j][k];
    }
    if(itr<maxWidth)res[rows-1][itr++]=' ';
}
*returnSize=rows;
```

```
    return res;
}
```

Time Complexity:

1. 1 ms
2. 0 ms
3. 4ms
4. 0 ms

Space Complexity:

1. 8.70 MB
2. 11.13 MB
3. 11.6 MB
4. 9.2 MB

Test Case Results

Sample Test Cases:

1.

```
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
strs =
["flower","flow","flight"]
Output
"fl"
Expected
"fl"
```

2.

```
Input
grid =
[[1,3,1],[1,5,1],[4,2,1]]
Output
7
Expected
```

7

3.

Input
a =
"11"
b =
"1"
Output
"100"
Expected
"100"

4.

Input
words =
["This", "is", "an", "example", "of", "text", "justification."]
maxWidth =
16
Output
["This is an","example of text","justification. "]
Expected
["This is an","example of text","justification. "]

Result: All Submitted successfully

Evaluation
Correctness: Yes

Time Complexity: 1ms

Space Complexity: 8.70 MB

Code Quality:

Code is readable

Proper loops and conditions used Memory allocation handled correctly

Plagiarism Check: No plagiarism detected

Marks / Grade
Total Marks: 10/10

Breakdown:
- Logic / Approach – 3 / 3
- Implementation (Code) – 3 / 3
- Test Cases Passed -2 / 2

- Documentation & Comments – 2 / 2

Leetcode Link:

https://leetcode.com/u/Mrunali_Jibhakate/